RadianceSwap: In-Process Multi-Model LLM Serving with Dynamic VRAM Sharing

The RadianceSwap Contributors
https://github.com/Code000-211/RadianceSwap
Technical report, November 2026 (code at `86d4503`)

***Abstract*—Serving several large language models from one GPU group usually means running one inference process per model and restarting processes as demand shifts. Each restart re-parses the checkpoint, re-lays out the weights and rebuilds tokenizer and execution state, and a request for one model can wait for another model's longest generation. We present RadianceSwap, an extension of the Radiance inference engine that serves several models from a single process behind one OpenAI-compatible endpoint. Models are activated by their first request, deactivated when idle, and kept in host RAM in GPU-ready form, so that re-activation needs no parsing, relayout or disk read. Several models can be active at once: each runs as an independent runtime, a ticket-based sequencer gives all tensor-parallel collectives one group-wide order, and a group planner moves expert, KV-cache and static-weight memory between models at run time. On two AMD Radeon AI PRO R9700 GPUs (TP = 2), RadianceSwap adds no measurable overhead for a single model ($-0.3\,\%$ to $+0.1\,\%$), re-activates a dense 0.8B model in 2.1 s, keeps outputs bit-identical to single-model serving, and lets an idle MoE model hand $\sim$3 GiB per card to a busy one within 30 s. The approach is experimental and was evaluated on one system.**

***Index Terms*—LLM serving, multi-model inference, GPU memory management, mixture of experts, tensor parallelism**

I. Introduction

Operators of on-premise inference servers increasingly want to offer several models at once: a large mixture-of-experts (MoE) model for quality, a small dense model for latency, an embedding or image model beside them. GPU memory, however, rarely holds all of them. The common practice is to run one inference server per model behind a proxy that starts and stops server processes on demand, as in llama-swap [2]. Two costs follow from that design. First, every switch pays the full start-up path: checkpoint parsing, weight relayout and the construction of tokenizer, chat template and execution plans; for a 113.6 GiB MoE container this exceeds a minute. Second, when only one model fits at a time, a request for model B waits for model A's longest running generation, which is unbounded, unless running generations are aborted.

RadianceSwap removes both costs inside the Radiance engine [1]. A single process owns the HTTP listener and the GPU group; models come and go as *runtimes* within it. The design rests on three decisions, recorded as architecture decision records (ADRs) in the repository: switching happens in process (ADR 0001); several models may be active at the same time and share VRAM (ADR 0002); and active models run as separate, truly parallel runtimes whose collectives follow one group-wide order (ADR 0003).

This report makes the following contributions:

- A **model lifecycle** with a host-resident, GPU-ready *prepared store* that turns re-activation into an upload without parsing, relayout or disk I/O, together with a host-memory budget that refuses an activation before allocation instead of triggering the kernel's OOM killer (§III-B, §III-F).
- **Run-time VRAM sharing** between active models: a heat-driven group planner for expert memory, an elastic expert slab that returns memory physically, cross-model KV-cache lending, and streaming of static weights from host RAM when a model does not fit (§III-D).
- A **collective sequencer** that prevents cross-model deadlock of spinning all-reduce kernels on a tensor-parallel group (§III-E).
- An **evaluation** on two AMD GPUs covering overhead, activation latency, time to first token, concurrency, memory movement, and four defects found during acceptance (§V), and a comparison with process-swapping proxies (§VI).

Without the `--manifest` option, Radiance behaves exactly as before; everything described here applies only to manifest mode.

II. Background

a) Radiance: Radiance is a C++ inference engine with architecture and kernel plugins and a ROCm/HIP device layer [1]. It loads `.rad` containers or Hugging Face checkpoints, converts them into the kernels' packed layouts, and serves an OpenAI-compatible API. Its placement layer already supports tiered expert residency: a planner fills a fixed number of VRAM expert slots, and a mover promotes and demotes experts by observed routing *heat*, serving the rest from a pinned host pool.

b) Tensor parallelism: With tensor parallelism (TP) [5], each layer's weights are sharded across the ranks of a GPU group, and partial results are combined by an all-reduce after attention and MLP blocks. Radiance's all-reduce kernels busy-wait on a flag written by the peer rank. Two independent models issuing such collectives concurrently can therefore deadlock if the ranks disagree on the order.

c) Mixture of experts: MoE layers [6], [7] route each token to a small subset of experts. Expert popularity is skewed and changes with the workload, which makes experts the natural unit for moving memory between models: a cold expert can be served from host memory at low cost, as exploited by offloading systems [8]–[10].

III. Design

Fig. 1 shows the components. A JSON *manifest* names the GPU group, cache policies and the models with their per-model engine options; it replaces `--model`. Unknown keys are rejected, so a typo cannot become a silent default.

A. Process model and routing

HTTP listener, plugin and tool registries and the coordination of the GPU group belong to the process; a model runtime cannot close them (ADR 0001). The *ModelRouter* reads a request's `model` field *before* tokenization, so every request uses the tokenizer, chat template, grammar and context limit of its own model. A request without `model` goes to the default model; an unknown id yields HTTP 404. Requests for a model that is not active join that model's bounded queue (64 by default, HTTP 429 beyond). All concurrent first requests share a single activation, which runs on its own thread so that no HTTP thread builds an engine. Image and audio requests are routed the same way; video jobs and WebSocket sessions are refused with HTTP 501 in manifest mode, and LoRA adapters are rejected at start-up because an adapter belongs to one base model.

B. Model lifecycle

Each model moves through the states of Fig. 2. Only the default model is started with the process; every other model starts `cold`. A model is `active` only once all ranks are ready. Deactivation is triggered by an idle timeout or by VRAM pressure. A model is idle only if no request lies between routing and the end of its response, so a streaming response keeps it busy until its last chunk; *a running generation is never interrupted*. Deactivation discards KV, prefix-cache and recurrent state but keeps weights, tokenizer, configuration and expert heat.

Under VRAM pressure, a model that finds the cards full causes the router to deactivate the active model that has been idle the longest; a model that is generating is never chosen, and one request triggers at most one such round per configured model. A failed activation (insufficient VRAM or host memory, a bad container, a rank error) releases every partial allocation on every rank and leaves other models untouched; retries back off exponentially from 1 s to 60 s. A device error instead marks the whole GPU group unhealthy, since retrying cannot help.

a) Prepared weight store: The first activation of a model copies its weights, in the packed form the kernels read, into a process-owned *PreparedWeightStore*. Weights are never dequantized or requantized. The store holds anonymous process memory rather than references to the container's page-cache mapping, because the kernel may reclaim page cache at any time; only the store makes "the second activation reads no checkpoint bytes" true. A re-activation from the store is an upload through a pinned staging ring.

C. Memory hierarchy

Fig. 3 summarizes the tiers. The ratio between VRAM and PCIe bandwidth, roughly 20–30×, governs every trade-off below: anything read over PCIe during decoding is expensive, so host-resident weights are a bridge, not a steady state.

Algorithm 1 Expert budget per rank (GroupPlanner)

Require: models $i = 1..n$ with offer o_i, floor f_i, expert plane E_i, heat per unit

$P \leftarrow \sum_i o_i$; $\quad s \leftarrow \mathit{min_share} \cdot P/n$
for each model i **do**
  $k_i \leftarrow \min(\max(f_i, s),\ |E_i|)$ ▷ guaranteed minimum
end for
$R \leftarrow P - \sum_i k_i$
$U \leftarrow$ expert units of all models, by $w(u) = \frac{heat(u) \cdot bytes(u)}{slot(u)}$, desc.
for $u \in U$ **while** $R > 0$ **do**
  $i \leftarrow \text{owner}(u)$
  if $k_i + \mathit{slot}(u) \le c_i$ **then** ▷ $c_i = o_i$, or P if elastic
    $k_i \leftarrow k_i + \mathit{slot}(u)$; $\quad R \leftarrow R - \mathit{slot}(u)$
  end if
end for
return targets k_i ▷ ties share room by bytes

D. Sharing VRAM between active models

Each model is assigned a fixed *share* of the free VRAM of the group's tightest card, less headroom, divided by $\min(n, m)$ where n is the number of models and m is `max_active_models`. The share caps the model's static weights, KV cache and experts. Inside the shares, four mechanisms move memory at run time (Fig. 4).

a) Group planner: The expert bytes a model's share leaves are its *offer*; the group's expert pool P is the sum of the offers. Algorithm 1 computes per-rank targets. Every model keeps a minimum, the larger of its *expert floor* (what its host pool cannot hold) and a fraction of the equal share P/n. The remainder goes to the most valuable expert units of the whole group, where an expert's worth is its heat times its bytes per byte of slot it occupies, which makes models with different expert sizes comparable. Running models apply new targets only at step boundaries, by capping their mover's resident slots. A model alone in the group receives exactly its offer, so single-model behaviour is unchanged.

b) Elastic expert slab: Logical targets alone do not move memory: a shrinking model's slab would remain one reservation. On cards with virtual-memory management and in manifests with more than one model, the expert slab is therefore *elastic*. It is mapped in chunks of about 128 MiB; a model may be granted up to the whole pool, but growth is paid only from chunks no member holds, and the receiver is granted bytes only after the giver reports their release, so the sum of committed slabs never exceeds the pool. Heat of an idle model halves every 4 s and counts as cold after 20 s, so an idle model shrinks to its minimum. Every chunk is mapped on *fresh* virtual address space; §V-G explains why. Each rank reserves 8 TiB of address space per model, about 65,000 chunk changes, after which the slab degrades to logical shrinking.

c) Cross-model KV lending: A model's share fixes only its initial KV cache (its *carve*); its ceiling is the context length times the number of sequences. When running or waiting prompts need more blocks than the carve holds, the cache

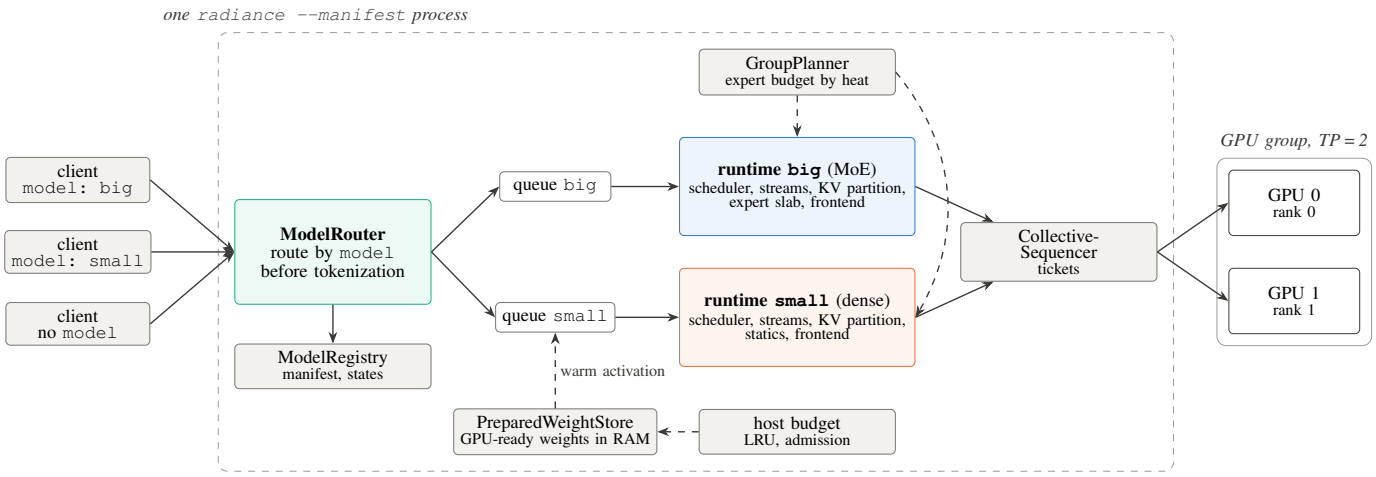


Fig. 1. Architecture of RadianceSwap. One process owns the listener and the GPU group. The router resolves the model before any tokenizer is touched; each active model is an independent runtime. Shared services decide expert budgets (GroupPlanner), order collectives (CollectiveSequencer) and hold deactivated models in GPU-ready form (PreparedWeightStore) under a host-memory budget.

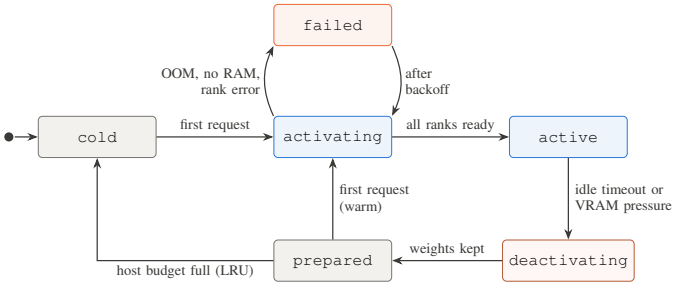


Fig. 2. Lifecycle of one model, as reported by `/radiance/models`.

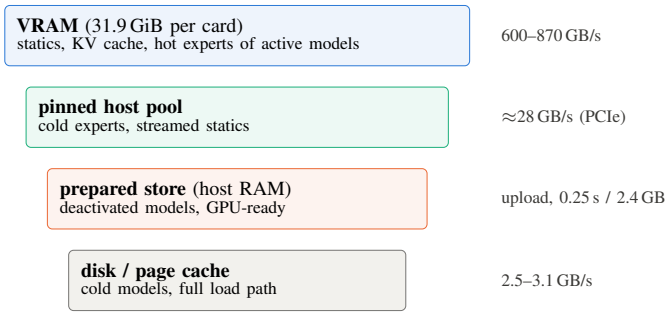


Fig. 3. Where a model's weights can live, with bandwidths measured on the test system.

claims chunks of 128–256 MiB (32,768 tokens per growing group) from the group's expert pool. The planner takes the bytes from the coldest experts of any model, each model keeping its minimum, and grants them once the pages are back. Running sequences are never touched, and a chunk is returned five seconds after demand ends.

d) Static-weight streaming: If a model's non-expert weights do not fit the VRAM left to it, it starts anyway: what does not fit stays in the pinned host pool and is read over PCIe, zero-copy for decode and through a staged ring for

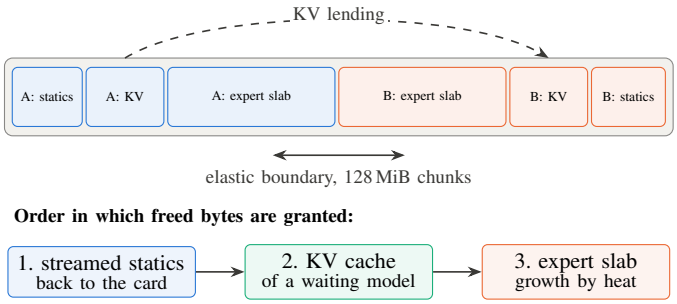


Fig. 4. Two active models on one card (schematic). Freed bytes go first to streamed static weights, then to the KV cache of a model with waiting prompts, and last to expert-slab growth.

prefill. Output is bit-identical. As soon as VRAM frees up, the streamed weights move to the card, and they are first in line for freed bytes (Fig. 4). Deterministic mode refuses streamed statics, because prefill without a stager would read weights in place over PCIe.

e) Heat warm-up: A deactivated model keeps its expert heat for the lifetime of the process. With a warm-up budget, the next activation ranks experts by that heat and places the hottest on the card at load time, instead of starting from the container's static profile.

E. Parallel runtimes and collective ordering

Each active model owns its streams, threads, contexts, pools and collective buffers per rank, and its forward passes run truly in parallel with other models' (ADR 0003). The danger lies in the all-reduce kernels, which spin on a peer flag: if rank 0 enters model A's all-reduce while rank 1 enters model B's, both spin forever. RadianceSwap applies the classical remedy of imposing a total order on resource acquisition [11]: a process-wide *CollectiveSequencer* assigns monotonic tickets on rank 0, and all other ranks issue their collectives in that exact sequence (Fig. 5). A device-side event chain enforces the order on the

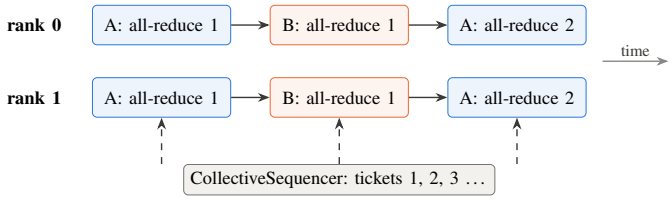


Fig. 5. All collectives of all models follow one ticket order, identical on every rank, so no rank can spin on model A while its peer has entered model B.

streams without stalling the host, and with a single registered engine the sequencer reduces to a relaxed atomic check.

The cost is that two models never communicate at the same time. Two busy models therefore share, rather than add, the communication capacity of the group; the benefit is that the second model is served without waiting for the first.

F. Host-memory budget and admission control

Host RAM is the binding constraint for large MoE models, because an active model's weights exist twice: on the card (or in its host pool) and in the prepared store. RadianceSwap books three kinds of host memory against `cache.host_budget_mib`: store copies, the host pools of active models (enlarged by streaming), and load staging. Page cache is never counted. When a booking does not fit, the longest-idle prepared model is dropped to `cold` (LRU [12]), then the store copy of an active model, and finally the activating model's own copy. After planning and *before* allocating, an activation checks its whole host bill against the budget, `MemAvailable`, all enclosing memory cgroups and a headroom; if it still does not fit, the activation is refused with HTTP 503 and nothing that runs is affected. Before this check, a streaming model beside a 138 GiB MoE ended in a kernel kill of the whole process. A practical sizing rule is

$$\text{RAM} \gtrsim \sum \text{store copies} + \sum \text{active host pools} + \text{staging} + \text{headroom}.$$

IV. Implementation

RadianceSwap is implemented in C++20 and HIP within the Radiance tree. The main components are the `ModelRegistry` (manifest parsing and per-model records), the `ModelRouter` (routing, queues, activation threads, idle and pressure deactivation), the `PreparedWeightStore` (`core/model/weight_store.h`), the `GroupPlanner` (`core/place/group_planner.cpp`), which hands each engine a byte budget that its existing planner and mover fill, so that there is no second placement algorithm, and the `CollectiveSequencer` (`core/runtime/collective_sequencer.cpp`). Enabling the lifecycle required splitting the engine's `serve()` into execution start/stop and independent HTTP serving, and turning stack-local bridges into a lifetime-managed per-model frontend whose leases keep tokenizer and response sinks valid across a deactivation.

Testing follows three seams. HTTP behaviour is tested through `Router::dispatch` with fake model runtimes (routing, single activation under concurrency, cancellation while waiting, idle timeout, LRU, rollback, metrics, LoRA rejection). The group planner is tested directly (heat allocation, minima, slot release, KV reclaim, streaming fallback). GPU acceptance runs on real hardware (§V).

TABLE I
SINGLE-MODEL OVERHEAD: UNMODIFIED ENGINE (MAIN) VS. RADIANCESWAP, `--model` MODE, TP = 2. THROUGHPUT IN TOK/S.

Model	Build	C1	C8	Prefill
Dense 0.8B	main	325–332	2032–2056	57,000–65,100
	RadianceSwap	326–333	2015–2039	57,300–64,700
MoE, determ.	main	73.8–77.7	129–151	4645–4866
	RadianceSwap	72.8–89.0	137–226	4697–5150

V. Evaluation

a) Setup: All measurements were taken on 2× AMD Radeon AI PRO R9700 (gfx1201, 31.86 GiB each) with TP = 2, a host with 48 vCPUs and 170 GiB RAM, and a build for gfx1201. The dense model is Qwen3.5-0.8B (2.4 GB of weights); the MoE model is Qwen3.8-Flash-Next, a 113.6 GiB container with 67.5 GiB of weights and a 47.7 GiB memory-mapped n-gram table. C1 denotes one request generating 256 tokens with `ignore_eos`; C8 denotes eight concurrent requests, aggregated. Unless stated otherwise each value is the min–max range over three repetitions. All numbers are taken from the repository's acceptance record (`docs/swap/gpu-acceptance.md`); none were re-measured for this report.

A. Overhead

Table I compares single-model serving before and after the changes. Dense C1 and C8 agree within 1 %; prefill varies by ±5 % from run to run in both builds. A manifest that holds one model matches `--model` within −0.3 % to +0.1 %. Outputs are identical to `--model` for greedy decoding, seeded sampling ($T = 0.8$, top-$p = 0.9$), streaming, a 2,421-token prompt and MTP speculation, for dense and MoE models, also while another model is active.

B. Activation latency

Fig. 6 shows activation latency. For the dense model, a warm activation takes 2.1 s (p50; p95 2.2 s), of which only 0.25 s is weight upload; the remaining ∼1.9 s are planning, all-reduce initialization, arena and kernel set-up. For the MoE model the store saves only 10 s of 70 s, because the store holds the 67.5 GiB of weights (≈6 s to upload) but the 47.7 GiB n-gram table is backed by page cache and is copied again at each activation; since the store displaces the page cache, that copy comes from disk. Placing the table on disk reduces the warm weight load to 6.3–7.6 s. Deactivation takes 0.7 s (dense) to 1.4 s (MoE). Switching between two models that are both already active costs no activation at all.

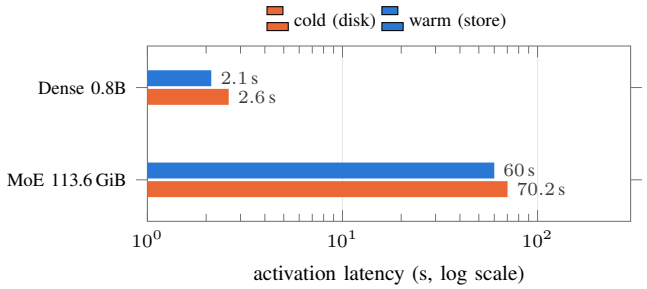


Fig. 6. Activation latency from disk and from the prepared store. Dense warm: p50 over 118 activations. The MoE warm activation is dominated by re-copying the mapped n-gram table; with `--ngram-placement disk` its weights load in 6.3–7.6 s.

TABLE II
TTFT DECOMPOSITION, DENSE 0.8B, 1,700-TOKEN PROMPT, P50 / P95 IN MS. THE PARTS SUM TO TTFT WITHIN 0.6 MS.

Case	Queue	Activation	Prefill	TTFT
model active	0.1 / 0.2	0	23.7 / 25.0	24.0 / 25.5
cold	0.5 / 0.8	1968 / 2007	37.9 / 38.1	2008 / 2045
prepared (warm)	0.4 / 0.8	2125 / 2227	28.8 / 29.5	2155 / 2256
32 clients, 8 seqs	1403 / 1461	0	27.1 / 103.6	1431 / 1490

C. Time to first token

Every response reports TTFT split into router wait for activation, scheduler queue and prefill, and `/metrics` exports the same parts as per-model histograms. Table II shows that for a model that is not active, activation is almost the whole TTFT, while for an overloaded active model the scheduler queue dominates. Activating a second model under that load took as long as without it, and its own queue stayed below 1 ms.

D. Concurrent models

Fig. 7 and Table III quantify the cost of collective ordering. Two busy dense models obtain about half the single-model decode rate each, so aggregate throughput stays near the single-model level; the gain is availability, not capacity. A second active but idle model costs about 10 % because execution tapes are bypassed while several engines are registered. With two MoE models, host RAM is saturated (160 GiB of 170 GiB in use), and part of the drop stems from page-cache pressure; the two causes were not separated.

E. Moving memory between models

Fig. 8 shows the elastic slab with two entries of the MoE model, each offering 10.07 GiB per rank from a pool of 20.20 GiB. Under load on m1 alone, m1 commits 13.1 GiB while m2 falls to 7.1 GiB; reversing the load reverses the split. A change of load reaches its target within 20–30 s, moving 128 MiB per step and 5 GiB within a few seconds. Across 7,736 samples under a continuously shifting load, the sum of committed slabs never exceeded the pool (before four fixes, 3,561 of 4,793 samples did, by up to 3.4 GiB). Greedy outputs of both models matched the reference after every phase. With

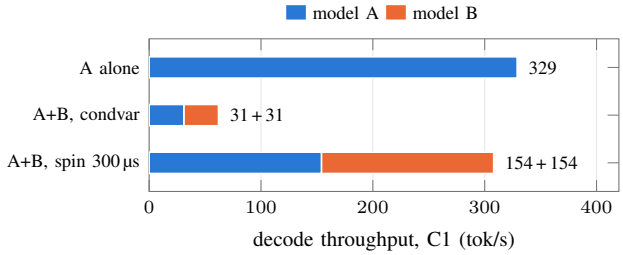


Fig. 7. Two dense models busy at once. With a condition-variable hand-off between rank threads, each model fell to 31 tok/s; spinning for 300 µs before sleeping restores the serial bound of about half the single-model rate each.

TABLE III
THROUGHPUT WITH TWO ACTIVE MODELS (TOK/S). "BUSY" MEANS THE OTHER MODEL IS UNDER LOAD AT THE SAME TIME.

Workload	Configuration	Throughput
Dense C1	alone	326–332
	other model active, idle	294–300
	other model busy	154 + 154
Dense C8	alone	2026–2029
	both busy	1287–1366 + 716–725
Dense prefill	alone	69,000–71,000
	beside busy model	36,000–47,000
MoE C1	alone (manifest, 1 model)	74.8–74.9
	beside busy dense model	59.3–59.4
Two MoE, C8	one busy / both busy	102 / 38.4 + 35.3

KV lending, a cache with a 1.9 GiB carve grew by 468–702 MiB per card under 12–16 concurrent long prompts while the other model's slab returned 2.8–3.5 GiB, and it shrank back 13–20 s after the load, with no lost sequence.

Fig. 9 quantifies static-weight streaming. Streaming 25.6 % of the dense model's statics costs 39 % of decode throughput, and 85.5 % costs 90 %; output remains identical, and throughput recovers within 1–2 s once VRAM frees up and the weights move back. An activation that finds the cards full succeeds in every tested case, which is the purpose of streaming: the new model answers immediately while it waits for memory.

a) Heat warm-up: For the MoE model, the early expert hit rate (first 64 steps, 246,817 routed experts) after a warm re-activation was 83.45 % without a warm-up budget and 89.12 % with 12 GiB, compared with 83.6 % after the cold start.

F. Robustness

Over 120 activate/deactivate cycles, VRAM returned to 266 MiB per card after every cycle; kfd queues, threads and file descriptors stayed constant, and host RSS grew by at most 17 MiB over the last 72 cycles. Failed activations (a model that can never fit, and a malformed container) were rolled back to the exact pre-activation free VRAM while a 3,000-token generation of another model continued undisturbed. With `max_active_models = 2` and three models, each request for the third model deactivated the longest-idle model and was served after 4.9–5.1 s. With `host_budget_mib` set too low, a second MoE activation was refused with HTTP 503 and no kernel kill.

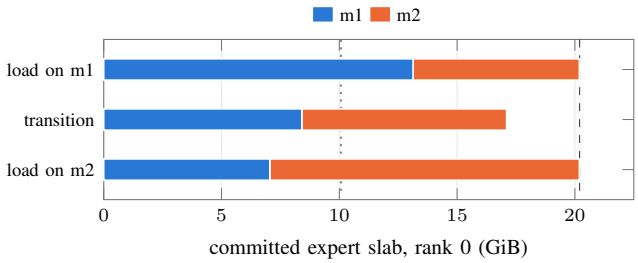


Fig. 8. Elastic expert slab with two Flash-Next entries. The loaded model grows about 3 GiB past its offer while the idle model shrinks to its minimum; the sum never exceeds the pool.

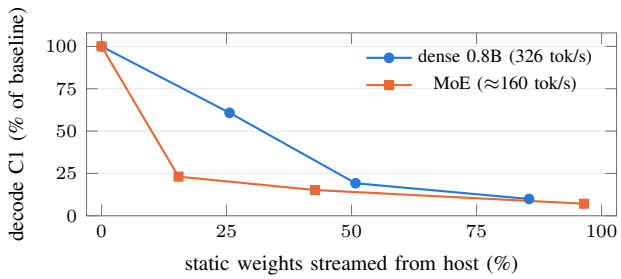


Fig. 9. Cost of streaming static weights over PCIe, normalized to the non-streaming baseline (midpoints of min–max ranges). For the MoE model the drop combines streamed statics and experts, which lose their VRAM slots at the same time.

G. Defects found during acceptance

Four defects illustrate where multi-model serving stresses assumptions of a single-model engine. (i) *Queue-id exhaustion*: each activation created ∼46 AQL streams whose ids were never reused; the 1,024-entry table overflowed at the 23rd activation, surfaced as a misleading out-of-memory error, and marked the group unhealthy. The table now has 2^{20} entries. (ii) *Wake-up latency*: the collective hand-off between rank threads used a condition variable whose sleep–wake cycle cost ∼0.5 ms, 48 times per decode step, reducing two concurrent models to 31 tok/s each; spinning for 300 µs first restored 154 tok/s. (iii) *Mis-counted capacity*: the store's capacity check included the memory-mapped n-gram table, which is never copied, so the MoE model was never prepared. (iv) *Virtual-address aliasing*: on the tested ROCm stack, re-mapping a previously unmapped virtual address had no effect, so the address kept pointing at its first physical page; once another model owned that page, two experts aliased and a request occasionally produced no finite logit. Mapping every chunk on fresh address space removed the error (30 load cycles with external observers, 5,776 requests, 0 errors).

VI. Related Work

a) Process-level model swapping: llama-swap [2] places a proxy in front of independent inference servers such as llama.cpp [3] and replaces the running server when a request names another model; operators declare which models may be co-resident. Ollama similarly loads models on demand and unloads them after a keep-alive period [4]. These systems are backend-agnostic and portable, which RadianceSwap is not. RadianceSwap's ADR 0001 considered exactly this design, an external router restarting one process per model with weights warm in the page cache, and rejected it for switch latency and the extra component. Table IV contrasts the two approaches. A combination is plausible but untested: since Radiance serves the OpenAI API, a RadianceSwap process could be one upstream of llama-swap. No benchmark compares the two on the same hardware; the latency advantage of warm in-process activation follows from the design but was not measured against llama-swap.

b) Multi-model serving systems: Clockwork [13] and PipeSwitch [14] reduce the cost of swapping DNN models on GPUs through predictable scheduling and pipelined transfer of layers. AlpaServe [15] shows that model parallelism enables statistical multiplexing of several models across devices, and MuxServe [16] multiplexes several LLMs spatially and temporally on shared GPUs. ServerlessLLM [17] accelerates cold starts with a loading-optimized checkpoint format and multi-tier storage. RadianceSwap shares the goals of fast activation and co-location but operates inside a single engine on a fixed tensor-parallel group, and adds run-time movement of expert, KV and static-weight memory between co-located models.

c) Offloading and memory management: FlexGen [9] and DeepSpeed-Inference [10] offload weights and KV cache to host memory to run models larger than GPU memory, and Eliseev and Mazur [8] cache MoE experts on the GPU while offloading the rest. PagedAttention [18] manages the KV cache in blocks to reduce fragmentation. RadianceSwap reuses Radiance's existing expert tiering and block-paged KV cache and extends them across model boundaries. S-LoRA [19] serves many adapters of one base model; RadianceSwap instead serves distinct base models and refuses adapters in manifest mode.

VII. Limitations and Future Work

The evaluation covers one system, one MoE model and small dense models; more than two models and MoE/MoE combinations under an explicit host budget were not tested. All models in a manifest share one TP size, so plugins that run only with TP = 1 cannot join a TP = 2 group, and the `devices` key is validated but not yet used to select cards. A cold model still reserves its VRAM share, which can make a large MoE ∼9 % slower even if the second model never activates (issue #56). A second active model costs ∼10 % even when idle. KV, prefix and recurrent state are discarded on deactivation; preserving them in host memory is future work. A long-run check of the elastic slab after the address-aliasing fix is still open (issue #58), as is lending one model's idle KV blocks to another model's expert slab.

VIII. Conclusion

RadianceSwap shows that multi-model serving can be moved from a process-swapping proxy into the inference engine itself.

TABLE IV
RADIANCESWAP COMPARED WITH LLAMA-SWAP, A PROCESS-SWAPPING PROXY (LLAMA-SWAP DETAILS FROM ITS README AND EXAMPLE CONFIGURATION, NOVEMBER 2026).

Aspect	llama-swap [2]	RadianceSwap
Architecture	external reverse proxy (Go) that starts and stops upstream server processes	in-engine, one process (C++/HIP)
Backends	any OpenAI- or Anthropic-compatible server (llama.cpp, vLLM, stable-diffusion.cpp, ...)	Radiance only (`.rad` or Hugging Face checkpoint)
Platforms	whatever the backend supports; binaries for Linux, macOS, Windows, FreeBSD	Linux, ROCm; evaluated on AMD gfx1201 only
Switch	stop process (graceful `unloadTimeout`, then kill), start another	tear down runtime; re-activate from the prepared store
Warm restart	depends on backend; at best weights in the OS page cache	GPU-ready weights in process memory; no parsing, relay-out or disk read
Co-residency	one model by default; `groups`, a `matrix` DSL with eviction costs, or operator-declared `capacity` pools	all models by default, bounded by memory budgets
VRAM split	fixed per process, declared by the operator	dynamic: group planner, elastic slab, KV lending, static streaming
Model does not fit	evict another model	start and stream from host RAM; deactivate idle models
Running generations	graceful unload with timeout (not examined here)	never interrupted
Multi-GPU	delegated to the backend	one TP group shared by all models, ordered collectives
Host memory	not managed	budget, LRU and admission control
Maturity	widely deployed, web UI, aliases, request filters, profiles	experimental, one test system

Keeping deactivated models GPU-ready in host memory makes re-activation an upload; ordering collectives makes truly parallel runtimes safe on a tensor-parallel group; and a heat-driven planner with an elastic slab, KV lending and static streaming lets VRAM follow the load. On the tested system this costs nothing for a single model, preserves bit-identical outputs, and turns the wait for a second model from an unbounded drain into seconds. The price is coupling to one engine and a narrow evaluation, which we hope testers on other hardware will broaden.

APPENDIX A
EXAMPLE MANIFEST

```
{
 "gpu_group": { "devices": [0, 1], "tp": 2 },
 "cache": {
   "idle_timeout_ms": 300000,
   "warmup_budget_mib": 12288
 },
 "models": [
   { "id": "big", "default": true,
     "path": "/models/big-moe.rad",
     "engine": ["--placement", "expert_tiered",
                "--max-model-len", "16384"] },
   { "id": "small",
     "path": "/models/small",
     "engine": ["--max-model-len", "16384"] }
 ]
}
```

Started with `radiance --manifest models.json --port 8000`; a request selects a model through its `model` field, and `/radiance/models` reports each model's state, VRAM and host memory. The full reference is `docs/GUIDE.md` §4.12.

REFERENCES

[1] Deadcode and the radiance contributors, "radiance," source repository, 2026. [Online]. Available: https://codeberg.org/StillDeadcode/radiance

[2] mostlygeek, "llama-swap," source repository, 2026. [Online]. Available: https://github.com/mostlygeek/llama-swap

[3] G. Gerganov *et al.*, "llama.cpp," source repository. [Online]. Available: https://github.com/ggml-org/llama.cpp

[4] Ollama, source repository. [Online]. Available: https://github.com/ollama/ollama

[5] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-LM: Training multi-billion parameter language models using model parallelism," arXiv:1909.08053, 2019.

[6] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *Proc. ICLR*, 2017.

[7] W. Fedus, B. Zoph, and N. Shazeer, "Switch Transformers: Scaling to trillion parameter models with simple and efficient sparsity," *J. Mach. Learn. Res.*, vol. 23, no. 120, 2022.

[8] A. Eliseev and D. Mazur, "Fast inference of mixture-of-experts language models with offloading," arXiv:2312.17238, 2023.

[9] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang, "FlexGen: High-throughput generative inference of large language models with a single GPU," in *Proc. ICML*, 2023.

[10] R. Y. Aminabadi, S. Rajbhandari, A. A. Awan, C. Li, D. Li, E. Zheng, O. Ruwase, S. Smith, M. Zhang, J. Rasley, and Y. He, "DeepSpeed-Inference: Enabling efficient inference of transformer models at unprecedented scale," in *Proc. SC*, 2022.

[11] J. W. Havender, "Avoiding deadlock in multitasking systems," *IBM Systems Journal*, vol. 7, no. 2, pp. 74–84, 1968.

[12] P. J. Denning, "The working set model for program behavior," *Commun. ACM*, vol. 11, no. 5, pp. 323–333, 1968.

[13] A. Gujarati, R. Karimi, S. Alzayat, W. Hao, A. Kaufmann, Y. Vigfusson, and J. Mace, "Serving DNNs like Clockwork: Performance predictability from the bottom up," in *Proc. OSDI*, 2020.

[14] Z. Bai, Z. Zhang, Y. Zhu, and X. Jin, "PipeSwitch: Fast pipelined context switching for deep learning applications," in *Proc. OSDI*, 2020.

[15] Z. Li, L. Zheng, Y. Zhong, V. Liu, Y. Sheng, X. Jin, Y. Huang, Z. Chen, H. Zhang, J. E. Gonzalez, and I. Stoica, "AlpaServe: Statistical multiplexing with model parallelism for deep learning serving," in *Proc. OSDI*, 2023.

[16] J. Duan, R. Lu, H. Duanmu, X. Li, X. Zhang, D. Lin, I. Stoica, and H. Zhang, "MuxServe: Flexible spatial-temporal multiplexing for multiple LLM serving," in *Proc. ICML*, 2024.

[17] Y. Fu, L. Xue, Y. Huang, A.-O. Brabete, D. Ustiugov, Y. Patel, and L. Mai, "ServerlessLLM: Low-latency serverless inference for large language models," in *Proc. OSDI*, 2024.

[18] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proc. SOSP*, 2023.
[19] Y. Sheng, S. Cao, D. Li, C. Hooper, N. Lee, S. Yang, C. Chou, B. Zhu, L. Zheng, K. Keutzer, J. E. Gonzalez, and I. Stoica, "S-LoRA: Serving thousands of concurrent LoRA adapters," in *Proc. MLSys*, 2024.